

QA TEST PLAN

Web Application — Manual & Automation Testing

Prepared by: Brian Scott McCarthy, Senior QA Lead

Tools: Azure DevOps • Playwright • Selenium • Postman • Cypress

1. Introduction & Purpose

This Test Plan defines the overall quality assurance strategy, scope, objectives, resource requirements, schedule, and risk mitigation approach for web application testing at Learning Experts Group and Rooms To Go Inc. It governs both manual exploratory testing and automated regression suites managed in Azure DevOps.

All test artifacts — test plans, test suites, test cases, and defects — are tracked and reported in Azure DevOps. CI/CD pipeline integrations via GitHub Actions and Jenkins trigger automated suites on every pull request and nightly build.

2. Test Objectives

- Validate all functional requirements against acceptance criteria defined in Azure DevOps User Stories and Tasks.
- Ensure cross-browser compatibility across Chrome, Firefox, Safari, and Edge using Playwright cross-browser execution.
- Verify API contract integrity and back-end service reliability using Postman and REST Assured before UI testing begins (shift-left).
- Confirm data integrity via SQL validation against order management, learner completion, and inventory databases.
- Achieve a minimum of 85% automated regression coverage for all P1/P2 user journeys.
- Maintain zero P1 defect escapes to production and reduce overall defect escape rate sprint-over-sprint.

3. Scope

3.1 In Scope

- Functional UI testing — all web application modules and user-facing workflows
- API testing — REST endpoints via Postman collections and REST Assured automation
- Regression testing — full and partial regression suites executed per sprint and pre-release
- Cross-browser testing — Chromium, Firefox, WebKit (Playwright); Chrome, Firefox (Selenium Grid)
- Mobile responsive testing — iOS Safari, Android Chrome (BrowserStack integration)
- SQL back-end data validation — database record integrity post-transaction
- Performance smoke testing — page load benchmarks on critical paths
- Security smoke — authentication flows, session management, input sanitization

3.2 Out of Scope

- Full load/stress performance testing (handled by dedicated performance team)

- Hardware/firmware testing
- Third-party vendor system internal testing

4. Test Strategy

Testing Type	Approach	Tools	Environment
Functional / Manual	Exploratory + scripted test cases executed against acceptance criteria	Azure DevOps Test Plans	QA, Staging
Automation – UI	Playwright (TypeScript) framework; page-object model; parallel execution	Playwright, GitHub Actions	QA, Staging
Automation – UI (Legacy)	Selenium WebDriver (Java) suite; TestNG + Maven; Selenium Grid 8 envs	Selenium, Jenkins	Multi-env
API Testing	Postman collections + REST Assured Java automation; contract & regression	Postman, REST Assured	QA, Staging
Regression	Full regression pre-release; partial per sprint on impacted paths	Playwright + Azure DevOps	Staging
Cross-Browser	Multi-browser parallel runs via Playwright and Selenium Grid	Playwright, Selenium Grid	QA
Mobile	Responsive UI validation; device emulation + BrowserStack	Playwright, BrowserStack	QA
Data Validation	SQL queries validating DB state post-transaction/API call	SSMS, pgAdmin	QA DB
Security Smoke	Auth flows, session tokens, CSRF, input sanitation spot checks	Manual + Postman	Staging

5. Azure DevOps Test Management Structure

All test artifacts are organized in Azure DevOps under the following hierarchy:

ADO Artifact	Purpose	Naming Convention
Test Plan	One plan per sprint / release. Contains all test suites for that cycle.	TP-[AppName]-Sprint[##]-[Year]
Static Test Suite	Groups test cases by feature area (e.g. Authentication, Checkout).	TS-[Feature]-[Type] e.g. TS-Auth-Functional

ADO Artifact	Purpose	Naming Convention
Requirement-Based Suite	Auto-linked to ADO User Stories; cases trace to acceptance criteria.	Linked to US-##### in ADO
Query-Based Suite	Dynamic suite pulling all active P1 cases or all failed cases for triage.	QBS-P1-Active / QBS-Failed-Sprint##
Test Case	Individual test scenario with steps, expected results, and data.	TC-[#####]-[Short Description]
Bug / Defect	Logged directly from failed test run; linked to test case + user story.	BUG-[#####]-[Short Description]

6. Entry & Exit Criteria

6.1 Entry Criteria

- Development team has completed feature build and deployed to QA environment.
- Acceptance criteria are documented on the Azure DevOps User Story.
- Test cases authored and reviewed in Azure DevOps Test Plan for the sprint.
- Test data provisioned and environment smoke test passed.

6.2 Exit Criteria

- 100% of planned test cases executed.
- All P1 and P2 defects resolved and verified; P3/P4 defects triaged with accepted risk sign-off.
- Automated regression suite passing at 95%+ in CI pipeline on staging branch.
- Test summary report published in Azure DevOps and reviewed by Product Owner.
- Go/No-Go decision documented by QA Lead.

7. Defect Management

Severity	Definition	SLA to Fix	ADO Priority Tag
P1 – Critical	Blocks core functionality; no workaround. Release blocker.	Same sprint / hotfix	Priority 1
P2 – High	Major feature broken; workaround exists but impacts UX significantly.	Current sprint	Priority 2
P3 – Medium	Feature partially broken; workaround available; moderate UX impact.	Next sprint	Priority 3
P4 – Low	Minor cosmetic or edge-case issue; minimal user impact.	Backlog grooming	Priority 4

Defects are logged in Azure DevOps with: reproduction steps, environment, browser/OS, screenshots or video, severity, linked test case, and linked User Story. All P1 defects trigger immediate Slack notification to Dev Lead and Product Owner.

8. Automation Framework Overview

8.1 Playwright (TypeScript) — Primary Framework

- Architecture: Page Object Model (POM); one class per page, methods per interaction.
- Execution: Parallel via Playwright workers; cross-browser (Chromium, Firefox, WebKit).
- CI Integration: GitHub Actions workflow triggers on PR and nightly schedule.
- Reporting: Playwright HTML report + Azure DevOps pipeline test results dashboard.
- Repos: github.com/BrianGator/Playwright-Test-Automation-2-w-TypeScript

8.2 Selenium WebDriver (Java) — Legacy/Parallel Suite

- Architecture: TestNG + Maven; Selenium Grid for multi-environment parallel execution.
- Coverage: 8+ application environments; regression + smoke suites.
- CI Integration: Jenkins pipeline; results published to Azure DevOps.
- Repos: github.com/BrianGator/Selenium-Test-Automation2-w-Java

8.3 Cypress (JavaScript) — Component & Integration

- Fast feedback for component-level and integration test scenarios.
- Intercept and stub network calls for isolated UI testing.
- Repo: github.com/BrianGator/Cypress-Automation-Testing-w-Javascript

8.4 Python PyTest — API & Data Validation Scripts

- Lightweight API assertion scripts and SQL data-validation automation.
- Repo: github.com/BrianGator/Python-PyTest-Scripting

9. Risks & Mitigation

Risk	Likelihood	Impact	Mitigation
Environment instability blocking test execution	Medium	High	Daily environment smoke test; Slack alerts on failures; fallback to local Docker env.
Test data corruption or unavailability	Medium	High	Isolated QA DB with seeded data scripts; refresh procedure documented in ADO wiki.
Automation suite	Low	High	Playwright auto-wait strategy; retry

Risk	Likelihood	Impact	Mitigation
flakiness masking real failures			logic (max 1); flaky tests flagged and quarantined within 24h.
Late requirement changes invalidating test cases	High	Medium	ADO traceability ensures impacted cases are flagged; re-estimation added to sprint planning.
Insufficient regression coverage for release	Low	High	Coverage metrics reviewed in sprint review; 85% automation target enforced.

10. Team & Responsibilities

Role	Name	Responsibilities
Senior QA Lead	Brian Scott McCarthy	Test strategy, automation architecture, ADO test plan management, Go/No-Go sign-off, stakeholder reporting.
Manual QA Engineers	QA Team	Test case execution, exploratory testing, defect logging, regression execution support.
Dev Lead	Development Team	Defect resolution, environment deployments, CI/CD pipeline maintenance.
Product Owner	Product Team	Acceptance criteria definition, defect priority decisions, Go/No-Go participation.
DevOps Engineer	DevOps Team	CI/CD pipeline configuration, environment provisioning, Docker/Kubernetes management.